Diploma in Computer Science & Information Technology

SBTE, Bihar — Semester IV

Theory of Computation

Course Code: 2418402

Unit-wise Detailed Notes

UNIT 1: Introduction to Theory of Computation

1.1 Alphabet, Languages, and Grammar

Alphabet (Σ)

An alphabet is a finite, non-empty set of symbols. It is commonly denoted by the Greek letter Sigma. Examples: Binary alphabet = {0,1}; English alphabet = {a,b,...,z}; DNA alphabet = {A,C,G,T}.

String

A string (or word) is a finite sequence of symbols from an alphabet. The empty string (string of length 0) is denoted by epsilon. The length of string w is denoted |w|. String concatenation: if x="ab" and y="cd", then xy="abcd".

Language

A language L over an alphabet is any set of strings over that alphabet. Examples: L = {0,1,00,11,000,...} = all binary strings; L = {a^n b^n | n >= 0} = strings with equal a's and b's. The empty language has no strings. The empty string language = {epsilon}.

Grammar

A grammar G is a 4-tuple G = (V, T, P, S) where: V = set of variables (non-terminals), T = set of terminals (alphabet), P = set of production rules, S = start variable.

1.2 Productions and Derivations

Production rules define how to replace variables with strings of variables and terminals. A derivation shows how a string is generated from the start symbol by applying production rules step by step.

Example: Grammar G with rules S -> aSb | epsilon. This generates: S => aSb => aaSbb => aaepsilonbb = aabb. The language generated is {a^n b^n | n >= 0}.

1.3 Chomsky Hierarchy of Languages

Noam Chomsky classified grammars into four types based on the restrictions on production rules. Each type corresponds to a class of languages and an automaton:

- **Type 3 — Regular Grammar:** Productions of the form A -> aB or A -> a. Recognized by Finite Automata (DFA/NFA). Examples: binary strings, identifiers.
- **Type 2 — Context-Free Grammar (CFG):** Productions of the form A -> gamma (any string of terminals/variables). Recognized by Pushdown Automata (PDA). Examples: balanced parentheses, a^n b^n.
- **Type 1 — Context-Sensitive Grammar (CSG):** Productions alpha A beta -> alpha gamma beta where |gamma| >= 1. Recognized by Linear Bounded Automata (LBA). More powerful than CFG.
- **Type 0 — Unrestricted Grammar:** No restriction on productions. Recognized by Turing Machines. Most powerful class. Includes all recursively enumerable languages.

Note: Every regular language is context-free; every context-free language is context-sensitive; every context-sensitive language is recursively enumerable. The reverse is NOT true.

UNIT 2: Regular Languages and Finite Automata

2.1 Deterministic Finite Automata (DFA)

A DFA is a 5-tuple M = (Q, Sigma, delta, q0, F) where: Q = finite set of states, Sigma = input alphabet, delta: Q x Sigma -> Q (transition function), q0 = start state, F = set of accepting/final states.

A DFA reads the input string one symbol at a time. At each step exactly one transition is possible. A string is accepted if processing ends in an accepting state.

2.2 Non-Deterministic Finite Automata (NFA)

An NFA is like a DFA but allows: multiple transitions for the same state and input, epsilon (empty) transitions (moving without consuming input), and no transition defined for some state-input pair.

An NFA accepts a string if AT LEAST ONE computation path leads to an accepting state.

NFA to DFA Conversion (Subset Construction)

Every NFA can be converted to an equivalent DFA. The states of the DFA correspond to subsets of NFA states. The DFA may have up to 2^n states for an n-state NFA.

2.3 Regular Expressions

A regular expression (RE) is a notation for describing patterns of strings (regular languages). Basic operations: Union (R1 | R2), Concatenation (R1 R2), Kleene Star (R*). Every regular expression corresponds to a regular language and vice versa.

Example: (0|1)*1 describes all binary strings ending in 1. a*b+ describes strings of zero or more a's followed by one or more b's.

2.4 Regular Grammar

A regular grammar is equivalent to DFA. Right-linear grammar: A -> aB | a (replace with terminal then optional variable). Left-linear grammar: A -> Ba | a. Every regular language can be described by a regular grammar.

2.5 Properties of Regular Languages

Regular languages are closed under: Union (L1 U L2), Intersection (L1 ∩ L2), Complement (~L), Concatenation (L1.L2), Kleene star (L*), Difference (L1-L2), Reversal.

2.6 Pumping Lemma for Regular Languages

Used to PROVE that a language is NOT regular. Statement: For any regular language L, there exists a pumping length p such that any string w in L with |w| >= p can be divided w = xyz where: |xy| <= p, |y| >= 1, and for all i >= 0, x(y^i)z is in L.

If you find a string that CANNOT be pumped, the language is not regular. Classic example: L = {a^n b^n} is not regular — pumping y within the a^p part breaks the balance.

2.7 Minimization of DFA

DFA minimization produces the smallest equivalent DFA. Method: (1) Remove unreachable states. (2) Mark distinguishable states — start by marking (accepting, non-accepting) pairs. (3) Iteratively mark more

pairs. (4) Merge indistinguishable states.

2.8 Mealy and Moore Machines

- **Moore Machine:** Output depends only on the current state. Output is produced when entering a state. Output function: lambda: Q -> output alphabet.
- **Mealy Machine:** Output depends on both current state and current input. Output is produced on transitions. More efficient — usually fewer states than equivalent Moore machine.

UNIT 3: Context-Free Languages and Pushdown Automata

3.1 Context-Free Grammars (CFG)

A CFG is a 4-tuple G = (V, T, P, S). Productions have the form A -> gamma where A is a single variable and gamma is any string of variables and terminals. CFGs describe a wider class of languages than regular grammars.

Example: Palindromes over {a,b}: S -> aSa | bSb | a | b | epsilon. Arithmetic expressions: E -> E+T | T, T -> T*F | F, F -> (E) | id.

3.2 Parse Trees and Ambiguity

A parse tree graphically represents the derivation of a string in a CFG. The root is the start symbol; leaves are terminals; internal nodes are variables.

A CFG is ambiguous if a string has more than one parse tree (or equivalently more than one leftmost derivation). Ambiguity is problematic in programming language design. Some CFLs are inherently ambiguous (no unambiguous grammar exists).

3.3 Normal Forms

Chomsky Normal Form (CNF)

Every production is either: A -> BC (two variables) or A -> a (single terminal). Useful for algorithms like CYK parsing. Any CFG can be converted to CNF.

Greibach Normal Form (GNF)

Every production is: A -> a alpha where a is a terminal and alpha is a string of variables (possibly empty). Each derivation step consumes exactly one terminal. Useful for constructing PDAs.

3.4 Pushdown Automata (PDA)

A PDA is like an NFA with an additional stack. It is a 7-tuple: (Q, Sigma, Gamma, delta, q0, Z0, F) where Gamma = stack alphabet, Z0 = initial stack symbol. The stack allows PDAs to handle non-regular languages like a^n b^n.

A PDA accepts a string either by final state or by empty stack. Non-deterministic PDAs (NPDA) are equivalent to CFGs.

3.5 Pumping Lemma for Context-Free Languages

For any CFL L, there exists p >= 1 such that any string w in L with |w| >= p can be written as w = uvxyz where: |vxy| <= p, |vy| >= 1, and for all i >= 0, u(v^i)x(y^i)z is in L.

Used to prove non-context-freeness. Example: L = {a^n b^n c^n} is not context-free — any pumping attempt on (v,y) that spans two types of letters breaks the equal count.

3.6 Closure Properties of CFLs

CFLs are closed under: Union, Concatenation, Kleene star, Reversal, Homomorphism. CFLs are NOT closed under: Intersection (in general), Complement. Exception: Intersection of a CFL with a regular language is always a CFL.

UNIT 4: Context-Sensitive Languages and Turing Machines

4.1 Context-Sensitive Grammars (CSG)

In a CSG, every production has the form alpha A beta -> alpha gamma beta where A is a variable, alpha and beta are strings, and |gamma| >= 1. The variable A is only replaced in the context of alpha and beta. This ensures that the length of the sentential form never decreases.

4.2 Linear Bounded Automata (LBA)

An LBA is a restricted Turing Machine that uses only the portion of tape occupied by the input string (linearly bounded). LBAs are equivalent to context-sensitive grammars — they recognize exactly the class of context-sensitive languages.

4.3 Turing Machines (TM)

A Turing Machine is a theoretical model of computation. It consists of: an infinite tape divided into cells, a read/write head, a finite set of states, and a transition function. At each step the TM reads a symbol, writes a symbol, moves left or right, and changes state.

Formal definition: TM = (Q, Sigma, Gamma, delta, q0, qaccept, qreject) where Gamma is the tape alphabet (including blank symbol B), and delta: Q x Gamma -> Q x Gamma x {L, R}.

4.4 Decidable and Recognizable Languages

- **Turing-Decidable (Recursive):** A language is decidable if a TM always halts on every input — accepting if the string is in the language, rejecting otherwise.
- **Turing-Recognizable (Recursively Enumerable):** A language is recognizable if a TM accepts strings in the language but may loop forever on strings NOT in the language.

Note: Every decidable language is recognizable, but NOT every recognizable language is decidable.

4.5 Variants of Turing Machines

- **Multi-tape TM:** Has multiple tapes and heads. More convenient but not more powerful — equivalent to standard TM.
- **Non-deterministic TM (NTM):** Can explore multiple computation paths simultaneously. Still equivalent in power to deterministic TM (but potentially exponentially faster).

• **TM as Enumerator:** A TM that generates (enumerates) all strings in a language one by one. A language is Turing-recognizable iff it is enumerable.

4.6 Unrestricted Grammars

Unrestricted grammars (Type-0) have no restrictions on production rules. They are equivalent in power to Turing Machines — they generate exactly the recursively enumerable languages.

UNIT 5: Undecidability

5.1 Church-Turing Thesis

The Church-Turing Thesis states that any function computable by an "effective procedure" (algorithm) can be computed by a Turing Machine. This is not a theorem but a philosophical principle linking informal notion of algorithm to formal TM computation.

Universal Turing Machine (UTM): A special TM that can simulate any other TM given its description as input. Forms the basis of modern stored-program computers.

5.2 Diagonalization and Reduction

Diagonalization is a proof technique to show that some languages are not Turing-recognizable. The Diagonalization Language Ld = {"M" | M does not accept "M"} is not Turing-recognizable.

Reduction: Problem A reduces to Problem B (A <=m B) if we can solve A using a solution to B. If B is decidable and A reduces to B, then A is decidable. Conversely, if A is undecidable and A reduces to B, then B is undecidable.

5.3 Rice's Theorem

Rice's Theorem: Any non-trivial semantic property of Turing-recognizable languages is undecidable. A property P is: Non-trivial if some TMs have it and some don't. Semantic if it depends on the language the TM recognizes, not its implementation.

Examples of undecidable properties: Does TM M accept the empty string? Does L(M) contain exactly one string? Is L(M) regular? Is L(M) = Sigma*?

5.4 Complexity Classes: P, NP, NP-Complete, NP-Hard

• **P (Polynomial Time):** Class of decision problems solvable by a deterministic TM in polynomial time O(n^k). Examples: Sorting, shortest path, primality testing.

• **NP (Non-deterministic Polynomial):** Class of problems for which a proposed solution can be VERIFIED in polynomial time. NP stands for "Non-deterministic Polynomial time." P is a subset of NP.

• **NP-Complete:** Problems that are in NP AND every NP problem reduces to them in polynomial time. Solving one NP-complete problem efficiently would solve ALL NP problems. Examples: SAT (Boolean satisfiability), 3-SAT, Traveling Salesman, Graph Coloring.

• **NP-Hard:** At least as hard as the hardest NP problems but may not be in NP themselves (may not be decidable or verifiable in polynomial time). Examples: Halting problem, Chess on n x n board.

Note: The famous P vs NP question asks whether P = NP. It is one of the seven Millennium Prize Problems and remains unsolved.
